Example: Separable or Entangled?

Given

$$|\psi\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Step 1: Identify coefficients

$$\alpha_{00} = \tfrac{1}{\sqrt{2}},\ \alpha_{01} = 0,\ \alpha_{10} = 0,\ \alpha_{11} = \tfrac{1}{\sqrt{2}}$$

Step 2: Quick test

$$\alpha_{00}\alpha_{11} = \tfrac{1}{2}, \quad \alpha_{01}\alpha_{10} = 0 \qquad \Rightarrow \text{Not equal}$$

Conclusion: State is **entangled**

Example: Tensor Product of Basis States

Compute

$$|0\rangle \otimes |0\rangle$$

$$= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = |00\rangle$$

Conclusion

- Clearly separable (already factorized)
- Both qubits fixed at 0

Example: Basis State and Superposition

Compute

$$|1\rangle \otimes \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

Step 1: Coefficients

$$(0, 0, \tfrac{1}{\sqrt{2}}, \tfrac{1}{\sqrt{2}})$$

Step 2: Quick test

$$0 \cdot \tfrac{1}{\sqrt{2}} = 0, \quad 0 \cdot \tfrac{1}{\sqrt{2}} = 0 \qquad \Rightarrow \text{Equal}$$

Conclusion: State is **separable**

Example: Tensor Product of Superpositions

Compute

$$\left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) \otimes \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}\right)$$

Result

$$|\psi\rangle = \frac{1}{2} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}$$

Step 1: Coefficients $(\frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2})$

Step 2: Quick test

$$\tfrac{1}{2} \cdot (-\tfrac{1}{2}) = -\tfrac{1}{4}, \quad (-\tfrac{1}{2}) \cdot \tfrac{1}{2} = -\tfrac{1}{4} \Rightarrow \text{Equal}$$

Conclusion

- State is **separable**

Example: Already in Tensor Product Form

Given

$$|\psi\rangle = |0\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Observation

- Already written as one tensor product

Conclusion

- State is **separable**

Example: Separable or Entangled?

Given

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Vector form

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Quick test

$$\alpha_{00}\alpha_{11} = \frac{1}{2}, \quad \alpha_{01}\alpha_{10} = 0 \Rightarrow \text{Not equal}$$

Conclusion

- State is **entangled**

Table of Contents

Matrices as Quantum Transformations

In classical linear algebra matrices transform vectors.
In quantum theory matrices transform state vectors too, but not every matrix is physically allowed.

General form

$$|\psi'\rangle = U|\psi\rangle$$

Constraint

- U must preserve normalization and reversibility.
- Therefore U must be **Unitary**.

This is a major difference from classical ML

- In neural networks we can use arbitrary weight matrices and non-linearities.
- In quantum evolution, physically allowed state transformations must obey stricter rules.

Introducing $U^\dagger$?

Definition

- $U^\dagger$ = conjugate transpose of U
- Take transpose and complex conjugate

Example

$$U = \begin{bmatrix} 1 & i \\ 2 & 3 \end{bmatrix} \Rightarrow U^\dagger = \begin{bmatrix} 1 & 2 \\ -i & 3 \end{bmatrix}$$

Key idea

- Generalization of transpose for complex matrices
- Needed because quantum states use complex numbers

Unitary Transformations

Definition

$$U^\dagger U = I$$

Interpretation

- columns (and rows) are orthonormal
- lengths and angles are preserved

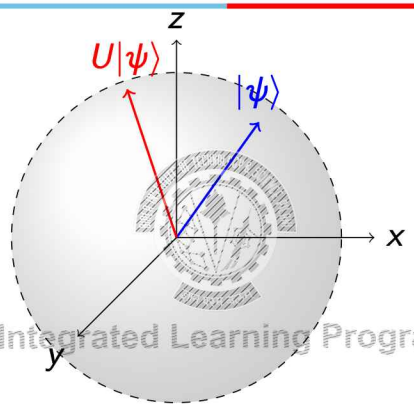
Why this matters

- normalization preserved ($\|\psi\| = 1$)
- evolution is reversible
- no information is lost

Geometric view

- rotation/reflection in complex space

Unitary as Rotation on Bloch Sphere



Key idea

- Unitary transformations rotate the state on the sphere
- Length stays the same $\Rightarrow$ normalization preserved
- Only direction (state) changes

Why Unitaries Preserve Norm

Take a normalized state

$$\langle\psi|\psi\rangle = 1$$

After applying U

$$|\psi'\rangle = U|\psi\rangle$$

Norm of new state

$$\langle\psi'|\psi'\rangle = \langle\psi|U^\dagger U|\psi\rangle = \langle\psi|I|\psi\rangle = \langle\psi|\psi\rangle = 1$$

Conclusion

- If U is unitary, then valid states remain valid.
- This is exactly why unitary matrices are the natural mathematical model for quantum gates.

Example: Checking Unitarity of a Simple Matrix

Consider

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Since entries are real

$$X^\dagger = X^\top = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Compute

$$X^\dagger X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

Therefore

- X is unitary.
- It swaps $|0\rangle$ and $|1\rangle$.

Example: A Matrix That Is Not Unitary

Consider

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

Compute

$$A^\dagger A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix} \neq I$$

Interpretation

- This matrix stretches some directions more than others.
- It changes norm.
- So it cannot represent an ideal quantum gate.

Lesson

- A valid matrix transformation in ordinary ML need not be valid in quantum theory.

Hadamard Matrix: A Central Example

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Action on basis states

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$H|1\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

What it does

- creates equal superposition from a basis state,
- introduces sign structure that later supports interference.

Verifying that Hadamard Is Unitary

Compute

$$H^\dagger H = H^\top H$$

since H is real.

$$H^\dagger H = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$= \frac{1}{2} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

Hence

- H is unitary.
- It preserves norm while changing basis.

Reversibility: A Core Difference from Classical Neural Networks

In a classical neural network we may apply affine transforms, ReLU, pooling, thresholding and many such operations are not reversible.

In ideal quantum evolution transformations must be reversible, so they must be unitary.

Why ReLU is not allowed as a quantum gate

- It is non-linear.
- It destroys information about negative values.
- It does not preserve norm.

Sequential Transformations: Matrix Composition

If two quantum operations are applied in sequence

$$|\psi'\rangle = U_2(U_1|\psi\rangle) = (U_2U_1)|\psi\rangle$$

Important point

- The rightmost matrix acts first.
- Matrix multiplication order matters.

If U_1 and U_2 are unitary

$$(U_2U_1)^\dagger(U_2U_1) = U_1^\dagger U_2^\dagger U_2 U_1 = U_1^\dagger I U_1 = I$$

So

- the composition of unitaries is unitary.
- This is why circuits made from valid quantum gates remain valid overall.

From Hermitian to Unitary: Intuition

Hermitian operator

H (represents a measurable quantity)

What it should satisfy

- real eigenvalues $\rightarrow$ meaningful outputs
- defines the quantity we want to evaluate (cost / energy)

Generate evolution

$$U = e^{-iHt}$$

Roles

- Hermitian $\rightarrow$ defines **what we measure**
- Unitary $\rightarrow$ defines **how the state moves**

Summary

So far we have learned

- states are normalized vectors in Hilbert space,
- similarity is inner product,
- joint systems use tensor products,
- operations are unitary matrices.

What is still missing?

- measurement,
- interpretation as qubits and gates,
- circuit-level construction.

Why Each Concept Was Needed

Concept	Why it matters in QML
Complex vectors	Quantum amplitudes carry magnitude and phase
Hilbert space	Gives a valid geometric state space
Normalization	Ensures proper probability interpretation
Inner product	Measures overlap / similarity / kernel value
Bra-ket notation	Makes expressions for states and operators compact
Tensor product	Combines qubits; dimension grows exponentially
Unitary transformation	Represents physically valid reversible evolution

Next Module

- Next we will make these abstractions concrete through:
  - qubits,
  - quantum gates,
  - multi-qubit gates,
  - circuit model.

BITS Pilani

Thank you :)

Quantum Machine Learning

Module 4: Quantum Foundations

BITS Pilani

Prof. Neha Vinayak

Where We Are (Quick Recap)

From Module 3, you already know:

- A quantum state is a vector: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$
- States live in a Hilbert space
- Superposition: system can exist in multiple basis states
- Tensor product: combining multiple qubits

But one big question remains:

Core Question

How do we **use these states to perform computation?**

Table of Contents

From States to Computation

A quantum state by itself is **static**.

To perform computation, we need to:

- Transform states
- Extract useful information

Key idea of quantum computation: $|\psi'\rangle = U|\psi\rangle$

- U = transformation (operator)
- A sequence of U's = computation

Analogy:

- Classical ML: layers transform features
- Quantum: operators transform states

Operators: What Are They?

An operator is a **rule that changes a quantum state**.

Mathematically: $U : \mathcal{H} \to \mathcal{H}$

Interpretation:

- Matrix acting on a vector
- Input state $\to$ transformation $\to$ new state

Example (Bit Flip):

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- $|0\rangle \to |1\rangle$
- This is the Pauli-X operation

Types of Operators in Quantum Systems

1. Observables (Measurement Operators)

- Used to extract information from a system
- Associated with measurable quantities
- Output is classical (0/1 or expectation values)

2. Unitary Operators (Evolution Operators)

- Used to transform quantum states
- Define the computation process
- Always reversible

Key distinction:

- Observables → give output
- Unitaries → perform computation

Quantum Evolution vs Measurement

Quantum evolution is deterministic:

$$|\psi(t)\rangle = U\,|\psi(0)\rangle$$

Measurement is probabilistic:

- Same state can produce different outcomes
- Output depends on amplitudes

Example:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

- Measure → 0 or 1 with equal probability

Key Insight

Computation happens during evolution; randomness appears at measurement

Evolution vs Measurement: The Full Picture

Two distinct processes in quantum computation:

Evolution (unitary)	Measurement
$\|\psi'\rangle = U\|\psi\rangle$	$P(i) = \|\langle i\|\psi\rangle\|^2$
Deterministic	Probabilistic
Reversible	Irreversible
No information lost	Collapses the state
Happens inside the circuit	Happens at the output

Table of Contents

Why Revisit Qubits?

Shift in Perspective

In Module 3 we studied what a qubit **is**. Now we study what we can **do** with it.

Module 3 view - representation:

- $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$
- State space, superposition, normalization
- Bloch sphere as a geometric picture

Module 4 view - computation:

- A qubit is an object we **actively manipulate**
- Goal: change amplitudes, introduce phase, control interference
- Tool: quantum gates acting as rotations on the Bloch sphere

Qubit as a Computational Object

Qubit = **object we manipulate using operators**

Goal of computation:

- Change amplitudes α, β
- Introduce phase relationships
- Control interference between states

Example:

- Start: $|0\rangle$
- Apply $H \rightarrow$ creates superposition
- Apply $Z \rightarrow$ changes phase (not probability directly)

Computation = controlled manipulation of amplitudes and phase

Table of Contents

Quantum vs Classical Gates

Classical gates:

- AND, OR, NOT
- Irreversible
- Work on bits (0 or 1)

Quantum gates:

- Always reversible (unitary)
- Work on amplitudes and phase
- Can create superposition and interference

Key Difference:

- Classical $\rightarrow$ manipulate values
- Quantum $\rightarrow$ manipulate probability amplitudes

Pauli-X Gate: Bit Flip Operation

Action:

- $|0\rangle \rightarrow |1\rangle$
- $|1\rangle \rightarrow |0\rangle$

Matrix form:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Bloch sphere view:

- Rotation of π radians around X-axis

Insight:

- Similar to classical NOT gate
- But still reversible

Pauli-Y Gate: Combined Bit and Phase Flip

Matrix form:

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Action:

- $|0\rangle \rightarrow i\,|1\rangle$
- $|1\rangle \rightarrow -i\,|0\rangle$

Bloch sphere view:

- Rotation of π radians around the Y-axis

How it differs from X and Z:

- X flips the bit: $|0\rangle \leftrightarrow |1\rangle$
- Z flips the phase: $|1\rangle \rightarrow -|1\rangle$
- Y does **both simultaneously**, with an extra factor of i

Pauli-Z Gate: Phase Flip Operation

Action:

- $|0\rangle \rightarrow |0\rangle$
- $|1\rangle \rightarrow -|1\rangle$

Important:

- Does NOT change measurement probabilities directly

Bloch sphere view:

- Rotation around Z-axis

Key insight:

- Phase changes affect future computation (interference)

S Gate: Quarter-Turn Phase Gate

Matrix form:

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$$

Action:

- $|0\rangle \rightarrow |0\rangle$ (no change)
- $|1\rangle \rightarrow i\,|1\rangle$ (phase shifts by 90)

Relationship to Z:

$$S^2 = Z$$

Applying S twice gives you Z. S is the "square root" of Z.

Bloch sphere view:

- Rotation of $\frac{\pi}{2}$ radians around the Z-axis
- Z rotates π, S rotates $\frac{\pi}{2}$ — half as much

T Gate

Matrix form:

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$$

Action:

- $|0\rangle \to |0\rangle$ (no change)
- $|1\rangle \to e^{i\pi/4}|1\rangle$ (phase shifts by 45)

Relationship to S and Z:

$$T^2 = S \quad T^4 = Z \quad T^8 = I$$

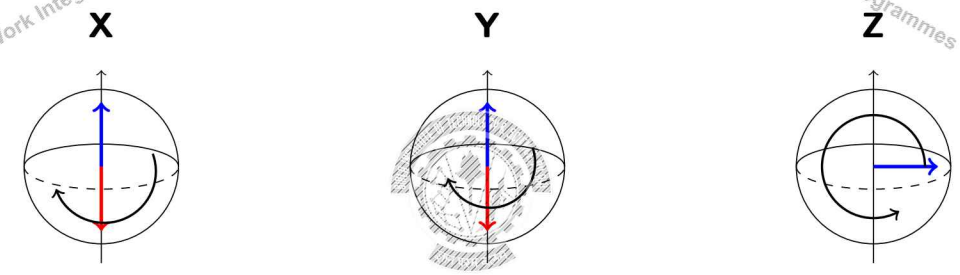
T is the "square root" of S and the "fourth root" of Z.

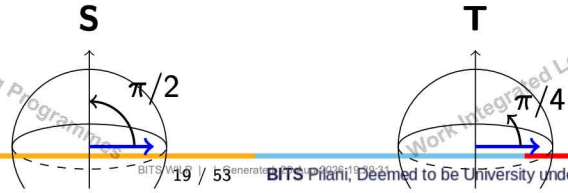
Bloch sphere view:

- Rotation of $\frac{\pi}{4}$ radians around the Z-axis

T + H + CNOT = **universal gate set**

Single-Qubit Gates on the Bloch Sphere





Single-Qubit Gates: Summary

Gate	Action on $\|1\rangle$	Bloch rotation	Role
X	$\|1\rangle \to \|0\rangle$	π around X	Bit flip
Y	$\|1\rangle \to -i\|0\rangle$	π around Y	Bit + phase flip
Z	$\|1\rangle \to -\|1\rangle$	π around Z	Phase flip
S	$\|1\rangle \to i\|1\rangle$	$\frac{\pi}{2}$ around Z	Quarter phase
T	$\|1\rangle \to e^{i\pi/4}\|1\rangle$	$\frac{\pi}{4}$ around Z	Eighth phase
H	$\|1\rangle \to \frac{1}{\sqrt{2}}(\|0\rangle - \|1\rangle)$	X+Z axis	Superposition

Hierarchy of phase gates:

Hadamard Gate: Creating Superposition

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Action:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Interpretation:

- Converts definite state → superposition
- Places qubit on equator of Bloch sphere

Why important?

- Enables parallel exploration of states
- Foundation for interference-based algorithms

What Happens After Hadamard? (Think)

Start with: $|0\rangle$

Apply:

- $H \rightarrow$ creates superposition
- $Z \rightarrow$ changes phase

Question:

- Does probability change after Z?

Answer:

- No immediate change
- But future operations will behave differently

Insight:

- Computation depends on phase evolution

Rotation Gates and Learning

Rotation gates provide continuous control:

$$R_y(\theta) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$$

Interpretation:

- Rotate qubit on Bloch sphere
- θ controls transformation

Connection to ML:

- θ = learnable parameter
- Neural networks $\rightarrow$ weights
- Quantum circuits $\rightarrow$ rotation angles

Learning = finding the right rotations

Bloch Sphere: Gates as Rotations

- Gates act as rotations on the Bloch sphere
- $R_x \rightarrow$ rotate around X-axis
- $R_y \rightarrow$ rotate around Y-axis
- $R_z \rightarrow$ rotate around Z-axis

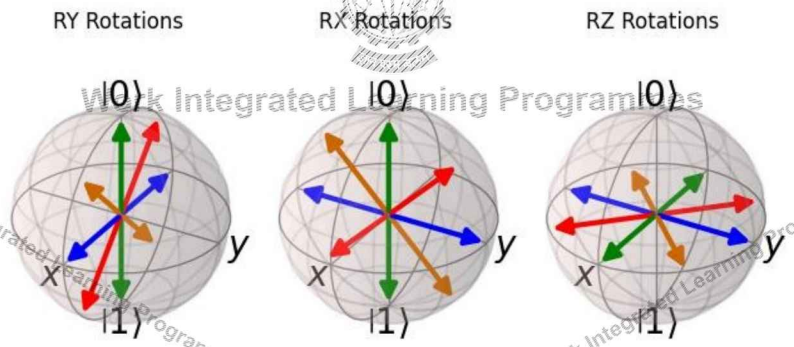


Table of Contents

Why Multi-Qubit Systems Are Powerful

2 classical bits

- One state at a time: 00, 01, 10, or 11
- n bits $\rightarrow$ describes 1 configuration

2 qubits

- All states simultaneously
- n qubits $\rightarrow 2^n$ amplitudes at once (Quantum Parallelism)

Where the power comes from:

- A separable state has only $2n$ amplitudes
- An entangled state has 2^n amplitudes
- This gap is a source of quantum computational advantage

Key Takeaway

Multi-qubit systems are powerful because quantum computation acts across all qubits at once.

Classical vs Quantum Correlation

Classical correlation:

- Example: two coins always show same value
- Either both heads or both tails

Quantum entanglement:

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

- System is not in a definite state before measurement
- Correlation appears only upon measurement

Key difference:

- Classical $\rightarrow$ hidden definite values
- Quantum $\rightarrow$ no definite value until measured

Measurement in Entangled Systems

Equal superposition (Bell state)

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Measure first qubit:

- $P(0) = \frac{1}{2}$
- $P(1) = \frac{1}{2}$

Each qubit individually looks random.

Unequal superposition (general)

$$\frac{1}{\sqrt{3}}|00\rangle + \sqrt{\frac{2}{3}}|11\rangle$$

Measure first qubit:

- $P(0) = \frac{1}{3}$
- $P(1) = \frac{2}{3}$

Probabilities are unequal

Why Multi-Qubit Gates?

Can single-qubit gates create entanglement?

No.

Entanglement requires interaction.

Single-qubit gates only

- Each qubit evolves independently
- No correlation between qubits
- No entanglement

Multi-qubit gates

- Qubits interact directly
- Correlations can be created
- Entanglement is possible

CNOT Gate

The most important two-qubit gate: CNOT (Controlled-NOT)

Operates on two qubits:

- **Control qubit** - determines the action (1 - Flip, 0 - do nothing)
- **Target qubit** - gets flipped or left alone

Examples:

Input	Output	Reason
$\|10\rangle$	$\|11\rangle$	control = 1, target flips $0 \to 1$
$\|00\rangle$	$\|00\rangle$	control = 0, target unchanged
$\|11\rangle$	$\|10\rangle$	control = 1, target flips $1 \to 0$
$\|01\rangle$	$\|01\rangle$	control = 0, target unchanged

Universal Gate Set

We now have all three essential building blocks:

H	T	CNOT
Creates superposition	Controls phase	Creates correlations
Puts qubit into both states simultaneously	Adds precise 45 phase — enables fine interference	Introduces interaction between qubits

Why each one is necessary:

- H alone → superposition but no phase control
- H + T → rich single-qubit states but no entanglement
- H + T + CNOT → full quantum computation

Creating Entanglement: Step by Step

Building a Bell state from scratch:

Step 0 — Start: $|00\rangle$ (both qubits at 0, separable)

Step 1 — Apply Hadamard to qubit 1:

$$|00\rangle \xrightarrow{H\otimes I} \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \quad \text{(still separable)}$$

Step 2 — Apply CNOT (qubit 1 = control, qubit 2 = target):

$$\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad \text{(entangled)}$$

Table of Contents

Quantum Circuits: From Gates to Computation

So far:

- Gates transform quantum states

Now: How do we build a computation?

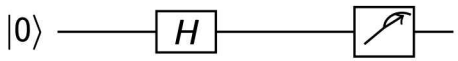
Key Idea

A quantum circuit is a **sequence of gates applied over time**

Pipeline:

- Prepare input state
- Apply sequence of gates
- Measure output

Example 1: Single Qubit Circuit



Step-by-step:

- Start: $|0\rangle$
- Apply H $\rightarrow$ superposition
- Measure $\rightarrow$ 0 or 1 with equal probability

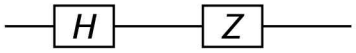
Key idea:

- Circuit defines probability distribution over outputs

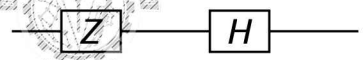
Example 2: Order Matters

Consider two circuits:

Circuit A



Circuit B



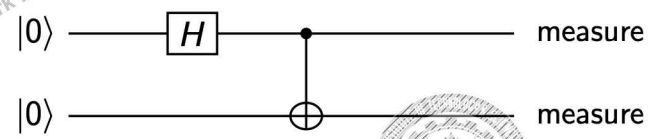
Observation:

- Both use same gates
- But final states are different

Insight:

- Quantum gates are order-sensitive

Example 3: Entanglement Circuit



Step-by-step:

- H $\rightarrow$ creates superposition
- CNOT $\rightarrow$ creates entanglement

Output:

- 00 or 11 (correlated outcomes)

Practice: Compute the Circuit Output

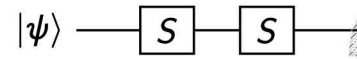
Circuit (starting from $|0\rangle$):

1. What is the state after each gate?
2. What is the final output state?
3. What are the measurement probabilities of $|0\rangle$ and $|1\rangle$?

Practice: Circuit Equivalence

Are the following two circuits equivalent?

Circuit A



$S^2|\psi\rangle$

Circuit B

$Z|\psi\rangle$

1. Test both circuits on $|0\rangle$ and $|1\rangle$.
2. Compute S^2 and compare with Z.
3. Are they equivalent? Justify.

Circuits as Functions

A quantum circuit defines a mapping:

$$\text{Input state} \rightarrow \text{Output probability distribution}$$

Important:

- Output is not a single value
- It is a distribution over possible outcomes

Key idea:

- Computation = shaping probability distribution

Putting It All Together

- Qubits store information
- Gates transform states
- Multi-qubit gates create correlations
- Circuits combine everything into computation

Final insight:

- Quantum computation = controlled evolution of probability amplitudes

Table of Contents

From Quantum Circuits to Machine Learning

So far:

- We built quantum circuits using gates
- Circuits transform input states into output distributions

Now the key question:

Can we use quantum circuits as **models** for learning?

Answer:

- Yes - by making parts of the circuit adjustable

Quantum Machine Learning Pipeline

Classical ML:

- Input data → model → prediction

Quantum ML:

- Input data → quantum circuit → measurement outcomes

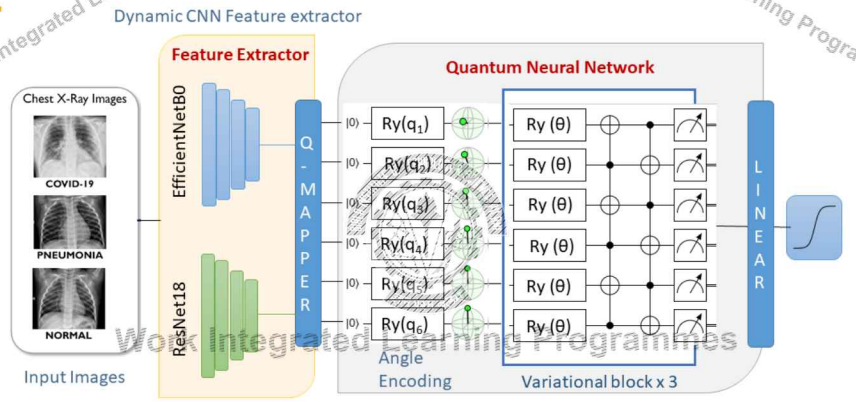
Mapping:

- Circuit = model
- Gates = transformations
- Measurement = prediction

Key difference:

- Output is a probability distribution

QML Pipeline



Key idea:

- Classical data → encoding → quantum circuit → measurement → prediction
- End-to-end hybrid quantum-classical pipeline

Where Does Learning Happen?

Some gates have adjustable parameters, like $R_y(\theta)$

Learning means:

- Changing θ to improve output

Analogy:

- Neural networks → weights
- Quantum circuits → rotation angles

Insight:

- Learning = finding the right transformations

Variational Quantum Circuits (Core Idea)

Variational circuit:

- Quantum circuit with tunable parameters

Structure:

- Data encoding
- Parameterized gates
- Measurement

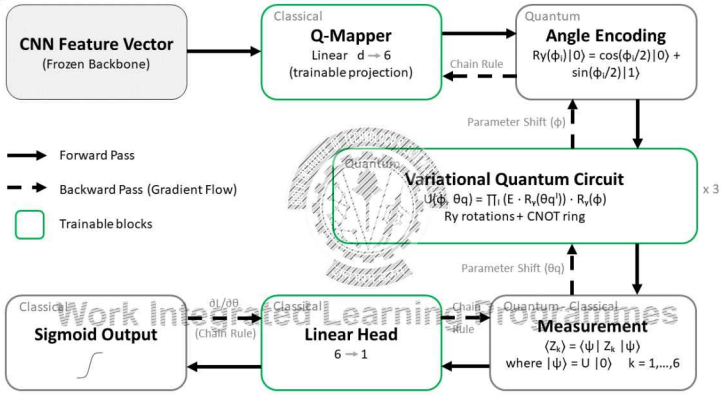
Goal:

- Adjust parameters to match desired output

Used in:

- Quantum neural networks

Hybrid Quantum-Classical Learning



- Forward pass → quantum + classical computation
- Backward pass → parameter updates via classical optimizer

Why Hybrid (Classical + Quantum)?

Quantum hardware limitations:

- No direct gradient computation
- Measurement noise

Solution:

- Use classical optimizer
- Update quantum parameters iteratively

Pipeline:

- Quantum circuit → output
- Classical computer → update parameters

Insight:

- Learning happens in a hybrid loop

What We Have Built So Far

- Qubits → store information
- Gates → transform states
- Multi-qubit systems → create correlations
- Circuits → define computation

Now:

- We can use circuits as learnable models

Big picture:

- Quantum computation → Quantum machine learning

What is Missing? (Next Module)

So far, we assumed:

- Input is already a quantum state

But in reality:

- Data is classical (images, text, signals)

Key Question

How do we convert classical data into quantum states?

Next:

- Data encoding
- Feature maps
- Quantum embeddings

Hands-On (IBM Quantum Composer)

In the next demo, you will:

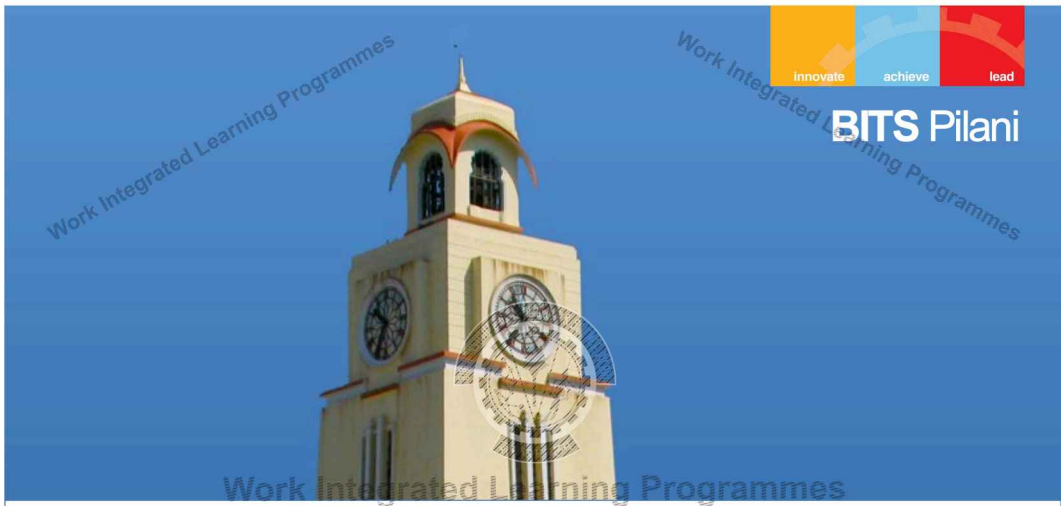
- Apply gates on real qubits
- Observe measurement outcomes
- Build simple circuits

What to focus on:

- How state changes after each gate
- Difference between X, H, and Z
- Effect of applying multiple gates

Goal:

- Build intuition: "What happens when I apply this gate?"



Thank you :)

Quantum Machine Learning

Module 5: Encoding

Prof. Neha Vinayak

BITS Pilani

Table of Contents

Why Data Encoding is Critical in QML

Central Question:

How do we represent classical data inside a quantum system?

Classical ML:

- Data already exists as vectors
- Directly fed into models

Quantum ML:

- Algorithms operate on **quantum states**
- Data must be transformed into $\psi(x)$

Key Insight

Encoding = Feature Mapping into Hilbert Space

Encoding as Feature Mapping

Core idea:

$$x \to |\psi(x)\rangle$$

- Classical data is mapped into a quantum state
- This is equivalent to a **feature transformation**

Connection to ML:

- Classical ML: $x \to \phi(x)$
- Quantum ML: $x \to |\psi(x)\rangle$

Key insight:

- Encoding defines the representation of data
- This directly affects learning performance

Encoding Defines Similarity

Similarity between two data points:

$$K(x, x') = |\langle \psi(x) | \psi(x') \rangle|^2$$

- Inner product measures overlap (Module 3)
- Measurement extracts probabilities (Module 4)

Interpretation:

- Encoding defines geometry of the data
- Different encodings $\Rightarrow$ different kernels

Takeaway:

- Encoding is not preprocessing
- It defines the learning problem itself

From Classical Data to Quantum Representation

In Classical Machine Learning:

- Data is represented as vectors:

$$x = [x_1, x_2, x_3, \ldots, x_N]$$

- Algorithms operate directly on these vectors
- Similarity is computed using dot products, distances, kernels

In Quantum Computing:

- Algorithms operate on **quantum states**, not vectors
- Data must be transformed into a valid quantum state

$$x \longrightarrow \psi(x) \in \mathbb{C}^{2^n}$$

Table of Contents

Amplitude Encoding: Concept

Definition:

$$\psi = \sum_{i=0}^{N-1} x_i i$$

Key Idea:

- Data stored in amplitudes of basis states
- Requires only $\log_2 N$ qubits

Example:

$$x = [x_0, x_1, x_2, x_3] \Rightarrow \psi = x_0 00 + x_1 01 + x_2 10 + x_3 11$$

Amplitude Encoding: Analysis

Encoding:

$$x \to |\psi(x)\rangle = \sum_i x_i |i\rangle$$

Requirement:

$$\sum_i |x_i|^2 = 1$$

Advantages:

- Exponential compression
- Uses fewer qubits

Limitations:

- State preparation is expensive
- Deep circuits required

Insight:

Solved Example: Amplitude Encoding

Given:

$$x = [1, 1]$$

Normalize:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Probabilities:

$$P(0) = \frac{1}{2}, \quad P(1) = \frac{1}{2}$$

Interpretation:

- Data stored in amplitudes
- Measurement reveals squared values

Amplitude Encoding: Numerical Example

Given:

$$x = [1, 1, 1, 1]$$

Step 1: Normalize

$$||x|| = 2 \Rightarrow x = \frac{1}{2}[1, 1, 1, 1]$$

Step 2: Quantum State

$$\psi = \frac{1}{2}(00 + 01 + 10 + 11)$$

Interpretation:

- Equal probability distribution across all basis states
- Equivalent to uniform superposition

Amplitude Encoding: Circuit Intuition

Goal: Prepare arbitrary quantum state from $0^{\otimes n}$

Key Challenge:

- Requires complex sequence of controlled rotations
- Depth grows exponentially with features

Insight:

Efficient in qubits, expensive in circuit preparation

Amplitude Encoding: Practice

Q1: Encode $x = [2, 0, 0, 0]$

Solution:

$$\psi = 00$$

Q2: How many qubits needed for 8 features?

Answer:

$$\log_2 8 = 3$$

Table of Contents

Basis Encoding: Concept

Definition:

$$x = (b_1, b_2, ..., b_n) \rightarrow b_1 b_2 ... b_n$$

Key Idea:

- Each classical bit $\rightarrow$ one qubit
- Direct mapping

Example:

$$x = [1, 0, 1] \Rightarrow 101$$

Basis Encoding: Properties

Advantages:

- Simple implementation
- No normalization required

Limitations:

- No superposition
- Limited expressivity
- Requires many qubits

Not suitable for complex ML tasks

Basis Encoding: Analysis

Representation:

$$x = [x_1, x_2, \ldots, x_n] \rightarrow |x_1 x_2 \ldots x_n\rangle$$

Interpretation:

- Each feature stored as a qubit
- Direct mapping of classical bits

Limitations:

- Requires one qubit per feature
- Cannot represent continuous data directly
- No feature interactions

Insight:

- Simple but low expressivity

Solved Example: Basis Encoding

Given:

$$x = [1, 0, 1]$$

Encoding:

$$|\psi(x)\rangle = |101\rangle$$

State vector:

$$|101\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Basis Encoding: Practice

Q1: Encode decimal 5

$$5 = 101 \Rightarrow 101$$

Q2: Encode $[0, 1, 1, 0]$

0110

Table of Contents

Angle Encoding: Concept

Definition:

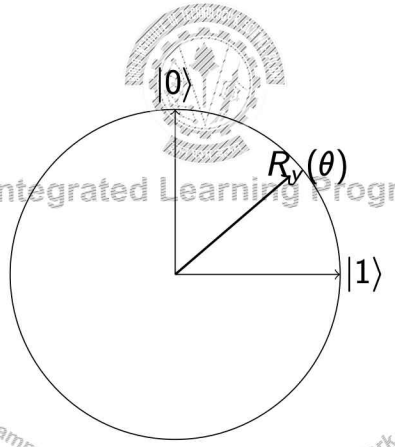
$$\psi(x) = R_y(x_1)R_y(x_2)..R_y(x_n)0^{\otimes n}$$

Key Idea:

- Data encoded as rotation angles
- Most widely used in QML

Angle Encoding: Geometric Intuition

- Each qubit = point on Bloch sphere
- Rotation changes quantum state
- Data controls rotation angle



Angle Encoding: Interpretation

Encoding:

$$|\psi(x)\rangle = \bigotimes_i R_y(x_i)|0\rangle$$

Interpretation:

- Each feature controls a rotation
- Data is mapped to geometry on Bloch sphere

Advantages:

- Efficient circuit
- Suitable for NISQ devices

Limitation:

- Limited feature interaction

Solved Example: Angle Encoding

Given:

$$x = [\pi/2]$$

Apply:

$$R_y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$$

Result:

$$R_y(\pi/2)|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Interpretation:

- Equal probability state
- Phase-free superposition

Angle Encoding: Numerical Example

Given:

$$x = [\frac{\pi}{2}, \pi]$$

Operations:

- Qubit 1: $R_y(\pi/2)$
- Qubit 2: $R_y(\pi)$

Result:

- Qubit 1 $\rightarrow$ superposition
- Qubit 2 $\rightarrow$ flipped state

Angle Encoding: Practice

Q1: What if $\theta = 0$?

$$0 \rightarrow 0$$

Q2: What if $\theta = \pi$?

$$0 \rightarrow 1$$

Q3: Why is this useful?

- Continuous data encoding
- Hardware efficient

Table of Contents

Hamiltonian Encoding: Concept

Definition:

$$\psi(x) = e^{-iH(x)t}0$$

Key Idea:

- Data controls Hamiltonian
- State evolves over time

Hamiltonian Encoding: Intuition

- Quantum system behaves like physical system
- Data defines "energy landscape"
- Evolution encodes structure

Insight

Used in quantum kernels and advanced QML models

Hamiltonian Encoding: Practice

Q1: What if $H = 0$?

$$\psi = 0$$

Q2: What happens as time increases?

- More evolution
- More complex state

Table of Contents

Comparison of Encoding Methods

Encoding	Qubits	Depth	Expressivity
Amplitude	$\log N$	High	High
Basis	N	Low	Low
Angle	N	Low	Medium
Hamiltonian	N	Medium	High

Observation:

- Tradeoff between qubits and circuit complexity
- No universally optimal encoding

Encoding as a Unitary Transformation

$$|\psi(x)\rangle = U(x)|0\rangle$$

- Encoding implemented using quantum gates
- $U(x)$ depends on data

Connection:

- Module 3: U is unitary
- Module 4: U is a circuit
- Module 5: $U(x)$ encodes data

Key idea:

- Data is transformed, not stored

Encoding Tradeoffs

Encoding	Qubits	Depth	Expressivity
Basis	High	Low	Low
Angle	Medium	Low	Medium
Amplitude	Low	High	High

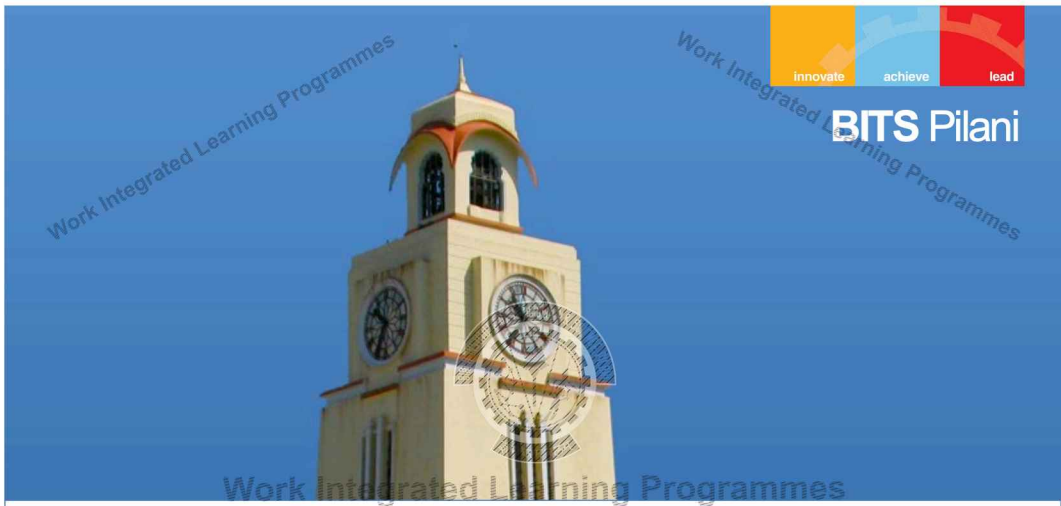
- No single best encoding
- Choice depends on hardware and task

Key Takeaways

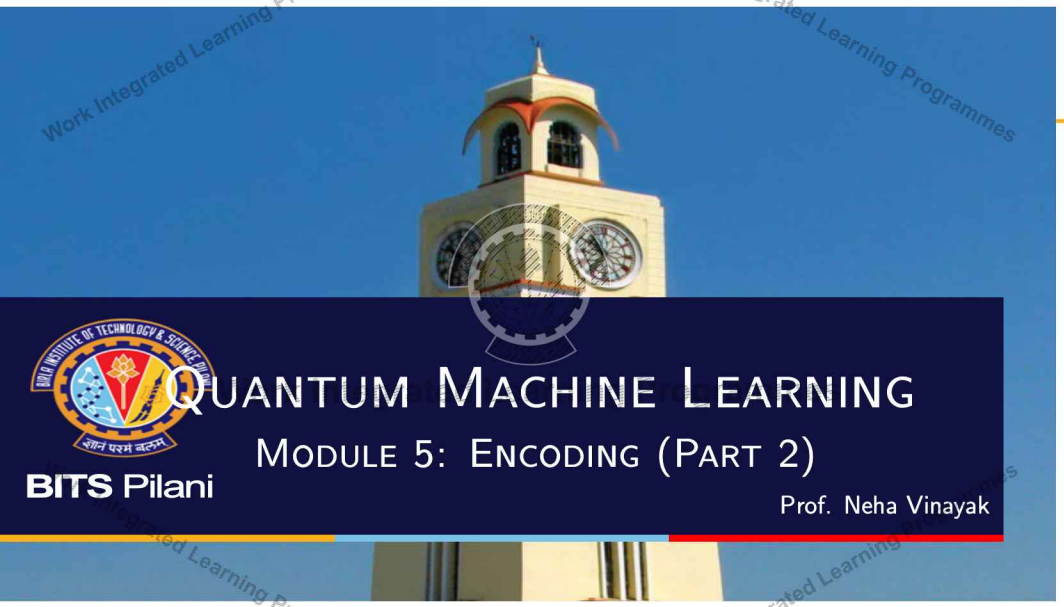
- Encoding is the foundation of QML
- Determines model performance and feasibility
- Different strategies suit different problems

Final Insight

Encoding = Feature Engineering in Quantum ML



Thank you :)



Quantum Machine Learning

Module 5: Encoding (Part 2)

Prof. Neha Vinayak

BITS Pilani

Table of Contents

Session 6 vs Session 7: What Changes?

Session 6 (Chapter 3.4):

- Studied different encoding techniques
- Focus on how to implement encodings using circuits
- Compared tradeoffs: qubits, depth, expressivity

Session 7 (Chapter 4):

- Focus on what encoding represents
- How data is stored and transformed in a quantum computer
- How encoding relates to learning

From **how to encode** $\rightarrow$ to **what encoding means**

Unifying View of Encoding

All encoding methods from Session 6 can be written as:

$$|\psi(x)\rangle = U(x)\,|0\rangle^{\otimes n}$$

Interpretation:

- x = classical input data
- $U(x)$ = data-dependent unitary transformation
- $|\psi(x)\rangle$ = encoded quantum state

Important Insight

Encoding is not just data loading - it is a transformation of data.

From Representation to Learning

Step 1: Representation

$$|x\rangle = |x_1 x_2 \dots x_n\rangle$$

Step 2: Transformation

$$|\psi(x)\rangle = U(x)|0\rangle$$

Key Idea:

- Representation stores data
- Transformation creates structure for learning

Big Picture

Learning happens because of how data is transformed, not how it is stored.

Table of Contents

Where Does Data Exist in a Quantum Computer?

In classical machine learning, data is naturally represented as vectors:$x = (x_1, x_2, ..., x_n)$ These values can be directly used in algorithms such as linear regression, SVMs, or neural networks.

In a quantum computer, algorithms do not operate on vectors directly.

Instead, all information must be encoded into quantum states:

$$|x\rangle = |x_1\rangle \otimes |x_2\rangle \otimes \cdots \otimes |x_n\rangle$$

Computational Basis States as Data Representation

The simplest and most fundamental way to represent classical data in a quantum computer is through **computational basis states**.

$$|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}, \quad |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}$$

For multiple bits, we use tensor products:$|x\rangle = |x_1 x_2 \dots x_n\rangle$

Example: $x = (1, 0, 1, 1) \Rightarrow |x\rangle = |1011\rangle$

Interpretation:

- Each qubit stores one classical bit
- No superposition or probability involved
- The state is deterministic

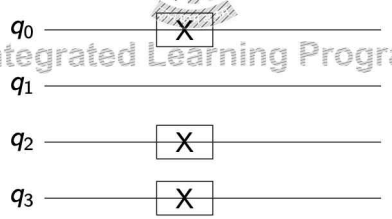
Circuit Realization of Basis State Representation

To prepare a computational basis state, we start from:

$$|0\rangle^{\otimes n}$$

and apply X (NOT) gates to flip selected qubits.

Example: Encode $x = 1011$



Important Distinction: Storage vs Encoding

It is important to distinguish between two uses of basis states:

1. **Representation:**
   - Basis states are used to **store classical data**
   - This is the default input format of quantum computers
   - No learning or transformation happens here
2. **Encoding:**
   - Basis encoding was treated as a **learning strategy**
   - Compared with amplitude, angle, Hamiltonian encoding

Limitations of Basis Representation for Learning

While basis states are essential for representing data, they are not suitable for machine learning tasks by themselves.

Why?

- No superposition → no parallel representation of information
- No interference → no constructive/destructive combination of features
- No entanglement → no feature correlations

Implication:

- Basis states do not provide a rich feature space
- They behave like classical data storage

If we only use basis states, quantum machine learning reduces to classical computation.

Why Representation Still Matters in QML

Even though basis states do not provide learning power, they play a critical role in quantum algorithms.

Role in Quantum Computation:

- Input to all quantum circuits - every circuit starts at $|0\cdots0\rangle$
- Required for controlled gates like CNOT
- Starting point for all encoding transformations

Role in Quantum ML Pipelines:

- Raw data → basis state → encoded state
- Basis states act as the **interface between classical and quantum worlds**

Table of Contents

From Representation to Transformation

In the previous section, we saw how classical data is represented as basis states: $|x\rangle = |x_1 x_2 \dots x_n\rangle$

Such states are too simple to enable learning.

To extract meaningful patterns, we need richer quantum states.

Goal of this section:

Transform a simple initial state into a complex quantum state that encodes data.

$$|0\rangle^{\otimes n} \xrightarrow{U(x)} |\psi(x)\rangle$$

General State Preparation Problem

Goal: given $x = (x_0, x_1, \dots, x_{N-1})$, find $U(x)$ such that:

$$U(x)|0\rangle = |\psi(x)\rangle = \sum_{i=0}^{N-1} \tilde{x}_i |i\rangle, \qquad \tilde{x}_i = \frac{x_i}{\|x\|}$$

Existence and Complexity - Shende, Bullock & Markov (2006)

Any n-qubit state can be prepared from $|0\dots0\rangle$ using at most $2^n - 1$ rotation gates and $2^n - 2$ CNOT gates.

This is both an existence guarantee and a tight upper bound. The question is not *whether* the circuit exists - but whether its $\mathcal{O}(2^n)$ depth is manageable in practice.

Connection to Amplitude Encoding

Amplitude encoding: $|\psi(x)\rangle = \sum_i x_i |i\rangle$

- How to construct this state physically
- What circuit implements this transformation

Let us reinterpret:

- Amplitude encoding = **state preparation problem**
- Requires building $U(x)$ explicitly

Important Insight

Amplitude encoding is not just a formula, it is a circuit construction problem. Data values appear as amplitudes, but amplitudes are outputs of rotation gates. State preparation is the problem of inverting that relationship: given the amplitudes you want, find the rotation angles that produce them.

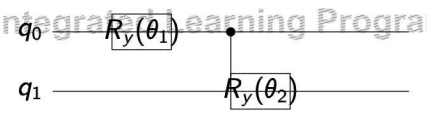
Two-Qubit Example: State Preparation

Target state:

$$|\psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle$$

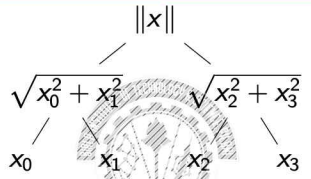
Strategy:

- First rotate qubit 1 to split probability mass
- Then conditionally rotate qubit 2



Key Idea: Amplitudes are distributed through hierarchical rotations.

Tree-Based Interpretation of State Preparation



Interpretation:

- Probability mass (sum of squared amplitudes) is split recursively
- Rotations correspond to branching decisions

Insight

State preparation is equivalent to constructing a probability distribution over basis states.

Complexity of State Preparation

Key Result:

- Preparing a general quantum state requires:

$$\mathcal{O}(2^n) \text{ gates}$$

Implications:

- Exponential scaling with number of qubits
- Deep circuits required

Contrast:

- Storage (basis state): $\mathcal{O}(n)$
- State preparation: $\mathcal{O}(2^n)$

State preparation is the main computational bottleneck in many QML algorithms.

Worked Example 1: Single-Qubit State Preparation

Data: $x = (3, 4)$

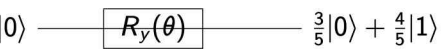
Step 1 - Normalise:

$$\|x\| = \sqrt{3^2 + 4^2} = 5 \quad \Rightarrow \quad |\psi\rangle = \frac{3}{5}|0\rangle + \frac{4}{5}|1\rangle$$

Step 2 - Find the rotation angle:

$$\cos\frac{\theta}{2} = \frac{3}{5}, \quad \sin\frac{\theta}{2} = \frac{4}{5} \Rightarrow \quad \theta = 2\arcsin\left(\frac{4}{5}\right) \approx 1.855 \text{ rad}$$

Step 3 - Circuit:



Circuit depth $= 1$, gate count $= 1$.

Worked Example 2: Two-Qubit State Preparation

Data: $x = (1, 2, 3, 4)$

Step 1 - Normalise:

$$\|x\| = \sqrt{1 + 4 + 9 + 16} = \sqrt{30}$$

$$|\psi\rangle = \frac{1}{\sqrt{30}}|00\rangle + \frac{2}{\sqrt{30}}|01\rangle + \frac{3}{\sqrt{30}}|10\rangle + \frac{4}{\sqrt{30}}|11\rangle$$

Step 2 - Rotation angle formula:
At every node, the angle is set by the fraction of that node's probability budget going to its left child:

$$\cos^2\frac{\theta}{2} = \frac{\text{probability of left child}}{\text{probability of current node}}$$

The $\sqrt{30}$ denominators cancel - you always work with ratios within a branch.

Worked Example 2: Computing the Angles

Applying the formula to $x = (1, 2, 3, 4)$:

θ_1 - root node, left child is the $|0\cdot\rangle$ branch:

$$\cos^2\frac{\theta_1}{2} = \frac{1^2 + 2^2}{1^2 + 2^2 + 3^2 + 4^2} = \frac{5}{30} = \frac{1}{6} \quad \Rightarrow \quad \theta_1 \approx 2.526 \text{ rad}$$

$\theta_2^{(0)}$ - inside $|0\cdot\rangle$ branch (carries 1, 2), left child is $|00\rangle$:

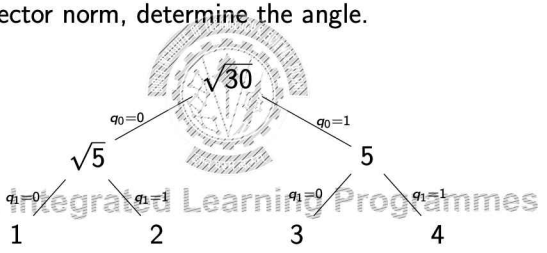
$$\cos^2\frac{\theta_2^{(0)}}{2} = \frac{1^2}{1^2 + 2^2} = \frac{1}{5} \Rightarrow \theta_2^{(0)} \approx 2.214 \text{ rad}$$

$\theta_2^{(1)}$ - inside $|1\cdot\rangle$ branch (carries 3, 4), left child is $|10\rangle$:

$$\cos^2\frac{\theta_2^{(1)}}{2} = \frac{3^2}{3^2 + 4^2} = \frac{9}{25} \quad \Rightarrow \quad \theta_2^{(1)} \approx 1.855 \text{ rad}$$

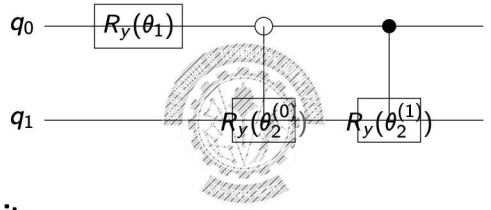
Worked Example 2: Probability Tree

Gate count $= 3 = 2^2 - 1$. Each angle is an independent subproblem - the branch values, not the full vector norm, determine the angle.



Root $\sqrt{30}$: total norm Level 1: $\sqrt{5}$ vs 5 - split by θ_1 Leaves: data values 1, 2, 3, 4

Worked Example 2: Circuit



Reading the circuit:

- $R_y(\theta_1)$ on q_0: splits probability between the two top-level branches
- Open circle: $R_y(\theta_2^{(0)})$ fires when $q_0 = |0\rangle$
- Filled circle: $R_y(\theta_2^{(1)})$ fires when $q_0 = |1\rangle$

Worked Example 3: Three-Qubit State Preparation

Data: $x = (1, 1, 1, 1, 1, 1, 1, 1)$ - uniform vector, 8 amplitudes, 3 qubits.

Step 1 - Normalise:

$$\|x\| = \sqrt{8} = 2\sqrt{2} \quad \Rightarrow \quad |\psi\rangle = \frac{1}{2\sqrt{2}} \sum_{i=0}^{7} |i\rangle$$

Step 2 - Rotation angle formula (same as Example 2):

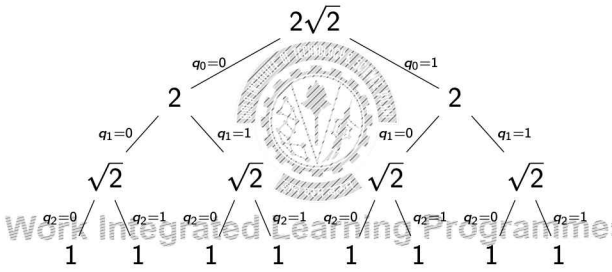
$$\cos^2 \frac{\theta}{2} = \frac{\text{probability of left child}}{\text{probability of current node}}$$

For 3 qubits, there are $2^3 - 1 = 7$ nodes in the tree - so 7 independent angles to compute: one at level 1, two at level 2, four at level 3.

Worked Example 3: Computing the Angles

θ_1	root, left carries $(1, 1, 1, 1)$	$\frac{4}{8} = \frac{1}{2} \Rightarrow \theta_1 = \frac{\pi}{2}$
$\theta_2^{(0)}$	inside $\|0\cdot\cdot\rangle$, left carries $(1, 1)$	$\frac{2}{4} = \frac{1}{2} \Rightarrow \theta_2^{(0)} = \frac{\pi}{2}$
$\theta_2^{(1)}$	inside $\|1\cdot\cdot\rangle$, left carries $(1, 1)$	$\frac{2}{4} = \frac{1}{2} \Rightarrow \theta_2^{(1)} = \frac{\pi}{2}$
$\theta_3^{(00)}$	inside $\|00\cdot\rangle$, left carries (1)	$\frac{1}{2} \Rightarrow \theta_3^{(00)} = \frac{\pi}{2}$
$\theta_3^{(01)}$	inside $\|01\cdot\rangle$, left carries (1)	$\frac{1}{2} \Rightarrow \theta_3^{(01)} = \frac{\pi}{2}$
$\theta_3^{(10)}$	inside $\|10\cdot\rangle$, left carries (1)	$\frac{1}{2} \Rightarrow \theta_3^{(10)} = \frac{\pi}{2}$
$\theta_3^{(11)}$	inside $\|11\cdot\rangle$, left carries (1)	$\frac{1}{2} \Rightarrow \theta_3^{(11)} = \frac{\pi}{2}$

Worked Example 3: The Probability Tree



Level 1: 1 rotation gate Level 2: 2 conditional rotations Level 3: 4 conditional rotations **Total: 7 gates**

Gate Count Across Examples

Example	Data dim N	Qubits n	Gates	$2^n - 1$
Ex. 1	2	1	1	1
Ex. 2	4	2	3	3
Ex. 3	8	3	7	7
General	N	$\log_2 N$	$N - 1$	$N - 1$

Pattern

Gate count $= N - 1 = \mathcal{O}(N) = \mathcal{O}(2^n)$. Saving qubits (from N to $\log_2 N$) costs gates (from n to $N - 1$).

Practice Problems

Problem 1. Given $x = (1, 1, 0, 2)$:

a. Normalise x and write the target quantum state $|\psi(x)\rangle$.
b. Draw the probability tree.
c. Compute θ_1, $\theta_2^{(0)}$, and $\theta_2^{(1)}$.
d. How many gates does the preparation circuit require?

Problem 2 (MCQ). Which of the following data vectors requires the *most* gates to amplitude-encode?

A. $x \in \mathbb{R}^4$
B. $x \in \mathbb{R}^8$
C. $x \in \mathbb{R}^{16}$
D. All require the same number of gates

Hint: recall the gate count formula from the table above.

Practical Challenges in NISQ Devices

In real quantum hardware:

- Limited coherence time
- Noise accumulation
- Restricted connectivity

Implication:

- Deep state preparation circuits are impractical
- Errors grow rapidly

Result:

- Exact amplitude encoding is rarely used in practice
- Approximate or shallow encodings preferred

Why This Matters for Quantum Machine Learning

Key takeaway:

- Many QML models assume efficient state preparation
- In practice, this assumption may not hold

Design Implication:

- Prefer encoding schemes with shallow circuits
- Example:
  - angle encoding
  - hardware-efficient ansatz

Research Direction:

- QRAM-based loading (theoretical)
- approximate state preparation

Table of Contents

Motivation: Beyond Arbitrary State Preparation

In the previous section, we saw that arbitrary state preparation:

- Requires complex, data-dependent unitary $U(x)$
- Has exponential circuit complexity in general
- Is difficult to implement on near-term quantum hardware

This raises a natural question:

Can we design structured, physically meaningful transformations to encode data?

Idea: use quantum dynamics itself as an encoding mechanism.

Instead of constructing $U(x)$ by decomposing it into elementary gates from scratch, we define it through time evolution under a Hamiltonian that depends on the data.

Encoding via Time Evolution: Formal Definition

In quantum mechanics, the evolution of a system is governed by a Hamiltonian H:

$$U(t) = e^{-iHt}$$

To encode data, we make the Hamiltonian depend on the input:

$$|\psi(x)\rangle = e^{-iH(x)}|0\rangle$$

Interpretation:

- Data x determines how the system evolves
- Instead of specifying the output state directly, we specify the *generator* of the transformation. So, data appears in the Hamiltonian that *produces* the state.

Physical Interpretation of Time Evolution Encoding

In physics:

- H represents the energy of the system
- e^{-iHt} describes how the system evolves over time

In encoding:

- $H(x)$ is a data-dependent generator
- Each input x produces a different Hamiltonian, driving the system along a different trajectory through Hilbert space
- The final state $|\psi(x)\rangle = e^{-iH(x)}|0\rangle$ is the endpoint of that trajectory

We are not storing data - we are letting data drive the evolution of a quantum system. The same data value, encoded through different Hamiltonians, produces entirely different states.

Eigenvalues and Eigenvectors: The Concept

When a matrix acts on a vector, direction and magnitude can both change. For certain special vectors, the matrix *only scales* - direction stays fixed:

$$H|v\rangle = \lambda|v\rangle$$

- $|v\rangle$: **eigenvector** - the direction H does not rotate
- λ: **eigenvalue** - the scaling factor

You already know two eigenvectors of Z: $Z|0\rangle = +1 \cdot |0\rangle$ $\quad$ $Z|1\rangle = -1 \cdot |1\rangle$
Z leaves both in the same direction - only flips the sign of $|1\rangle$.
What if a state is *not* an eigenvector?
$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ applied to Z:

$$Z|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$$

Output is a *different* state - Z rotated it, so $|+\rangle$ is **not** an eigenvector of Z.

Eigenvalues: Connection to Time Evolution

What does e^{-ixH} do to an eigenvector?

Since $H|v\rangle = \lambda|v\rangle$, evolution attaches a phase:

$$e^{-ixH}|v\rangle = e^{-ix\lambda}|v\rangle$$

Direction unchanged. Eigenvalue λ sets the phase; data x scales it.

For Z: eigenvalues $+1$ (for $|0\rangle$) and -1 (for $|1\rangle$):

$$e^{-ixZ}|0\rangle = e^{-ix}|0\rangle \qquad e^{-ixZ}|1\rangle = e^{+ix}|1\rangle$$

For a superposition: each component picks up its own phase:

$$e^{-ixZ} \cdot \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \tfrac{1}{\sqrt{2}}(e^{-ix}|0\rangle + e^{+ix}|1\rangle)$$

The *difference* between e^{-ix} and e^{+ix} encodes data x.

- Eigenstate ⇒ global phase only - unobservable, no encoding
- Superposition ⇒ relative phases - observable, data encoded

Simple Example: Single-Qubit Time Evolution

Consider a single-qubit Hamiltonian $H(x) = xZ$. The evolution operator is:

$$U(x) = e^{-ixZ} = \begin{bmatrix} e^{-ix} & 0 \\ 0 & e^{ix} \end{bmatrix}$$

Applied to $|0\rangle$:

$$e^{-ixZ}|0\rangle = e^{-ix}|0\rangle$$

This is a **global phase** - physically unobservable. The state remains $|0\rangle$ regardless of x. No encoding occurs because $|0\rangle$ is an eigenstate of Z.

For encoding to work, the initial state must be a superposition of eigenstates of H. Applying a Hadamard gate to $|0\rangle$ gives $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, after which e^{-ixZ} produces a *relative* phase between $|0\rangle$ and $|1\rangle$ that depends on x:

$$e^{-ixZ}\tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \tfrac{1}{\sqrt{2}}(e^{-ix}|0\rangle + e^{ix}|1\rangle)$$

Example: What if We Choose $H(x) = xY$?

Consider $H(x) = xY$. Same eigenvalues as Z (± 1), different eigenvectors, giving:

$$e^{-ixY} = \begin{bmatrix} \cos x & -\sin x \\ \sin x & \cos x \end{bmatrix}$$

Applied to $|0\rangle$ directly - no Hadamard needed, x controls the amplitudes:

$$e^{-ixY}|0\rangle = \cos(x)|0\rangle + \sin(x)|1\rangle$$

Different pixel intensities → directly different measurement probabilities.

xZ vs xY:

Hamiltonian	x controls	Needs Hadamard?
xZ	Phase gap (complex)	Yes
xY	Amplitude ratio (real)	No

Same data, different H ⇒ different feature map and different kernel. $R_y = e^{-ixY/2}$ is the most widely used gate: x enters probabilities directly.

Circuit Interpretation: $e^{-ixZ} = R_z(2x)$

The operator e^{-ixZ} is a standard rotation gate. By definition $R_z(\phi) = e^{-i\phi Z/2}$, so:

$$e^{-ixZ} = e^{-i(2x)Z/2} = R_z(2x)$$

The factor of 2 is the R_z convention - the argument is twice the evolution parameter.

Connection to Angle Encoding

Every Pauli rotation gate is a time-evolution encoding.
$R_y(x_i) = e^{-ix_iY/2}$, $R_z(x_i) = e^{-ix_iZ/2}$
Angle encoding is time-evolution encoding with single-qubit Pauli generators.

Extending to Multiple Features

For $x = (x_1, x_2, \ldots, x_n)$, the Hamiltonian is:

$$H(x) = \sum_i x_i H_i \qquad \Rightarrow \qquad U(x) = e^{-i\sum_i x_i H_i}$$

When the H_i commute (each acting on a different qubit):

$$e^{-i\sum_i x_i H_i} = \prod_i e^{-i x_i H_i}$$

The exponential factorises into independent single-qubit rotations. This is exactly **angle encoding** - depth 1, no entanglement between qubits.

When the H_i do not commute (e.g. H includes Z_iZ_j terms): the exponential does *not* factorise. Entangling gates are required - this is what the next section addresses.

Design Considerations

Choice of Hamiltonian determines how data influences the state. This is analogous to choosing the right kernel in a classical SVM.

Depth vs Expressivity:

- Shallow (single-qubit terms only) = angle encoding at depth 1
- Deeper (multi-qubit terms) = richer feature map, more noise on NISQ hardware

Feature Interactions:

- Single-qubit Pauli terms: product states, no entanglement
- Multi-qubit terms (Z_iZ_j, X_iX_j): entanglement, captures correlations between features - subject of next section

Table of Contents

Beyond Independent Features: Adding Interactions

Extend the Hamiltonian to include coupling terms:

$$H(x) = \underbrace{\sum_i x_i H_i}_{\text{individual}} + \underbrace{\sum_{i<j} x_i x_j H_{ij}}_{\text{pairwise interactions}}$$

The coefficient x_ix_j is large only when *both* features are large simultaneously

Example:

$$H(x) = x_1 Z_1 + x_2 Z_2 + x_1 x_2 Z_1 Z_2$$

Does $e^{-iH(x)}$ factorise?

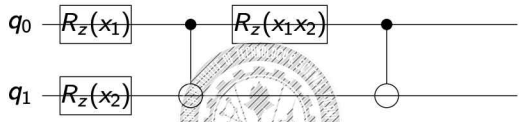
The coupling coefficient is x_1x_2

Similarly, Three-qubit terms add cubic interactions automatically.

The interaction order is controlled by how many qubits a Hamiltonian term couples.

Circuit Implementation of Interaction Terms

The full Hamiltonian $H(x) = x_1 Z_1 + x_2 Z_2 + x_1 x_2 Z_1 Z_2$ maps to the following circuit:



- $R_z(x_1)$, $R_z(x_2)$: individual feature terms
- CNOT–$R_z(x_1 x_2)$–CNOT: implements $e^{-ix_1 x_2 Z_1 Z_2}$
- After the ZZ gate, the state **cannot** be written as a product — qubit states depend on each other
- Amplitudes become functions of x_1 and x_2 *jointly*

Cost: each interaction term requires one CNOT pair. Pairwise interactions: $N(N-1)/2$ CNOT pairs.

Why Interactions Matter: Classical ML Parallel

In classical ML, to learn "high x_1 AND high x_2 predicts y", a linear model $y = w \cdot x$ fails.

In Hamiltonian encoding: the $x_i x_j$ interaction term is built into the physics. The quantum state automatically has amplitudes that depend on $x_i x_j$ - no manual engineering needed. **Three levers for expressivity:**

- **Interaction order**: single-qubit only = angle encoding; add pairwise terms for bilinear structure; three-body terms for cubic
- **Pauli type**: Z affects phases, Y mixes amplitudes, X flips basis states - choice determines Hilbert space directions explored
- **Repetitions**: re-encoding data multiple times with trainable layers between repetitions (data re-uploading, Module 7)

The cost: more interactions ⇒ more CNOT gates ⇒ more noise.

Table of Contents

Data Encoding as a Feature Map

Every encoding: $|\psi(x)\rangle = U(x)\,|0\rangle$ - a **feature map** from $\mathbb{R}^N$ into $\mathbb{C}^{2^n}$

- $|\psi(x)\rangle$ - **feature vector**
- $\mathbb{C}^{2^n}$ - **feature space**
- $U(x)$ - **feature mapping function**
- Dimension 2^n - exponentially larger than input N